DevOps Lab Assessment - Experiment 8

Continuous Deployment to Kubernetes via CI/CD Pipeline Automation

COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ASSESSMENT:**	Question 8 (5 Marks)
TOPIC:	CI/CD & Continuous Deployment to K8s	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Continuous Deployment (CD) automates the transition of validated code from source control directly to production Kubernetes clusters. The objective of this experiment is to architect and execute an automated CI/CD pipeline using **GitHub Actions** that compiles, tests, and packages a containerized application, publishes versioned images to an OCI registry, dynamically updates Kubernetes manifests via **Kustomize**, applies manifests to a cluster using kubectl, and automates rollback if health checks fail.

2. Continuous Deployment Pipeline Architecture

The automated delivery pipeline consists of three sequential workflow stages: (1) **Code Quality & Unit Testing**, (2) **Docker Multi-Stage Build & Security Scan** (Trivy), and (3) **Automated Kubernetes Manifest Application & Rollout Verification**.

Pipeline Phase	Trigger / Tool	Actions Performed	Output / Artifact
1. Continuous Integration	GitHub Actions / Pytest	Static linting (Flake8), runs automated unit test suite.	Test coverage report
2. Container Packaging	Docker Buildx / Trivy	Builds OCI image, scans CVE vulnerabilities, pushes to Docker Hub.	Tagged Image (SHA + v1.0)
3. Continuous Deployment	Kustomize & Kubectl	Patches image tag in deployment.yaml, executes kubectl apply.	Updated Pods in K8s
4. Rollout Verification	kubectl rollout status	Polls readiness probes; executes automated rollback if timeout occurs.	Zero-downtime release

3. Production CI/CD Pipeline Manifest (.github/workflows/deploy.yml)

```
// File: .github/workflows/deploy.yml

name: CI/CD Pipeline - K8s Automated CD on: push: branches: [main] jobs: build-and-push: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: docker/login-action@v3 with: { username: ${{ secrets.DOCKER_USER }}, password: ${{ secrets.DOCKER_TOKEN }} } - name: Build and Push Docker Image run: | IMAGE_TAG=devopsstudent/devops-microservice:${{ github.sha }} docker build -t $IMAGE_TAG -f src/app/Dockerfile src/app docker push $IMAGE_TAG deploy-to-k8s: needs: build-and-push runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: azure/k8s-set-context@v4 with: { method: kubeconfig, kubeconfig: ${{ secrets.KUBECONFIG_DATA }} } - name: Dynamically Update Manifests & Deploy run: | cd src/k8s kustomize edit set image devops-microservice=devopsstudent/devops-microservice:${{ github.sha }} kubectl apply -k . kubectl rollout status deployment/web-app-deployment --timeout=90s - name: Automated Rollback on Failure if: failure() run: | echo 'DEPLOYMENT FAILED! Initiating automated rollback...' kubectl rollout undo deployment/web-app-deployment
```

4. Step-by-Step CI/CD Execution & Deployment Logs

```
$ git commit -m 'feat: deploy release v1.0.1' && git push origin main
[main a48f102] feat: deploy release v1.0.1 2 files changed, 14 insertions(+), 2 deletions(-) To
github.com:devops-student/devops-lab-2.git 5b1a09c..a48f102 main -> main --> GitHub Actions Workflow Triggered: Run #42
(Commit: a48f102)
```

```
$ kubectl apply -k src/k8s/ kubectl rollout status deployment/web-app-deployment
configmap/app-config unchanged service/web-app-service unchanged deployment.apps/web-app-deployment configured Waiting for deployment "web-app-deployment" rollout to finish: 1 out of 3 new replicas have been updated... Waiting for deployment "web-app-deployment" rollout to finish: 2 out of 3 new replicas have been updated... Waiting for deployment "web-app-deployment" rollout to finish: 1 old replicas are pending termination... deployment "web-app-deployment" successfully rolled out
```

5. Post-Deployment Verification & Rollback Protection

```
$ kubectl get pods -l app=web-app -o wide
NAME READY STATUS RESTARTS AGE IMAGE web-app-deployment-7d9a102b-4x88k 1/1 Running 0 38s
devopsstudent/devops-microservice:a48f102 web-app-deployment-7d9a102b-9p21q 1/1 Running 0 38s
devopsstudent/devops-microservice:a48f102 web-app-deployment-7d9a102b-m001v 1/1 Running 0 38s
devopsstudent/devops-microservice:a48f102
```

6. Engineering Observations & Conclusion

Key Technical Takeaways

1. **Immutability via Git SHA:** Using ${{ github.sha }} tags eliminates caching ambiguities, guaranteeing that the exact tested commit is deployed.
2. **Declarative GitOps Delivery:** Kustomize modified the manifest programmatically without modifying source templates.
3. **Automated Self-Recovery:** The if: failure() trigger ensures that any deployment stalling past 90 seconds triggers kubectl rollout undo, restoring cluster stability automatically.